

Backend con Django Rest Framework



Aquí tienes la guía completa, en orden pedagógico, con archivos **completos** listos para copiar y pegar, rutas absolutas en Windows, notas de **NUEVO/EXISTENTE** y **cambios importantes**. Todo quedará funcionando en <http://localhost:8080/api/empleados> con MySQL y CORS para React (Vite) y Angular.

1) Preparar entorno (sin migraciones aún)

Meta: crear carpeta del proyecto, entorno virtual e instalar dependencias.

Carpeta base (Windows):

```
C:\Cursos\Python\rh_django
```

Abre terminal (PowerShell o CMD) en esa ruta y ejecuta:

1. Crear y activar entorno virtual

- *Qué hace:* crea un Python virtual env aislado para el proyecto y lo activa.

```
py -m venv .venv  
.\.venv\Scripts\activate
```

2. Actualizar pip

- *Qué hace:* evita errores por versiones viejas.

```
python -m pip install --upgrade pip
```

3. Instalar dependencias

- *Qué hace:* instala Django, DRF (API), PyMySQL (driver MySQL puro Python) y CORS.

```
pip install "Django>=5,<6" djangorestframework pymysql django-cors-headers
```

Aún **no** ejecutamos migraciones.

2) Crear proyecto y app

Desde C:\Cursos\Python\rh_django (con el venv activo):

4. Crear proyecto (en la carpeta actual)

- *Qué hace:* genera la estructura base de Django.

```
django-admin startproject rh_django .
```

5. Crear app “empleados”

- *Qué hace:* módulo donde pondremos modelo, serializer, vistas y urls del recurso Empleado.

```
python manage.py startapp empleados
```

Estructura resultante (parcial):

```
C:\Cursos\Python\rh_django
├─ manage.py
├─ empleados\
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations\
│   ├── models.py
│   ├── tests.py
│   └── views.py
└─ rh_django\
```

```
|— __init__.py
|— asgi.py
|— settings.py
|— urls.py
|— wsgi.py
```

3) Editar `settings.py` (archivo completo)

CAMBIOS IMPORTANTES

- Se agrega conexión **MySQL** (db: `recursos_humanos_db`, user `root`, pass `admin`).
- Se integra **Django REST Framework** (sin autenticación, sin paginación).
- Se configura **CORS** con `django-cors-headers`: apps, middleware, orígenes permitidos (Angular/React).
- Se registra la app `empleados`.

EXISTENTE — se reemplaza el contenido por este completo:

Ruta:

C:\Cursos\Python\rh_django\rh_django\settings.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'dev-secret-key-no-usar-en-produccion'
DEBUG = True
ALLOWED_HOSTS = ['localhost', '127.0.0.1']

# Apps del proyecto
INSTALLED_APPS = [
    # CORS primero en apps
    'corsheaders',

    # Django core
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # DRF
    'rest_framework',

    # App de dominio
    'empleados',
]

MIDDLEWARE = [
    # CORS debe ir lo más arriba posible y ANTES de SecurityMiddleware
```

```

    'corsheaders.middleware.CorsMiddleware',

    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'rh_django.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [], # No usamos templates en este API
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'rh_django.wsgi.application'

# --- MySQL (PyMySQL) ---
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'recursos_humanos_db',
        'USER': 'root',
        'PASSWORD': 'admin',
        'HOST': '127.0.0.1',
        'PORT': '3306',
        'OPTIONS': {
            'charset': 'utf8mb4',
            'init_command': "SET sql_mode='STRICT_ALL_TABLES'",
        },
        'CONN_MAX_AGE': 60, # Conexiones persistentes (dev)
    }
}

# Zona horaria recomendada (opcional)
LANGUAGE_CODE = 'es-mx'
TIME_ZONE = 'America/Mexico_City'
USE_I18N = True
USE_TZ = True

STATIC_URL = 'static/'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

```

```
# --- Django REST Framework ---
# Sin autenticación y sin permisos restrictivos (API abierta en dev)
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.AllowAny',
    ],
    # Render/parse JSON por defecto
    'DEFAULT_RENDERER_CLASSES': [
        'rest_framework.renderers.JSONRenderer',
    ],
    'DEFAULT_PARSER_CLASSES': [
        'rest_framework.parsers.JSONParser',
    ],
}

# --- CORS ---
CORS_ALLOWED_ORIGINS = [
    "http://localhost:4200", # Angular
    "http://localhost:5173", # React (Vite)
]

# (Opcional) Métodos permitidos explícitos (por claridad)
CORS_ALLOW_METHODS = [
    "GET",
    "POST",
    "PUT",
    "PATCH",
    "DELETE",
    "OPTIONS",
]
```

4) Habilitar PyMySQL como “MySQLdb” (compatibilidad)

CAMBIOS IMPORTANTES

- Evita problemas cuando Django espera MySQLdb.

EXISTENTE — reemplazar contenido (o añadir si viene vacío):

Ruta:

C:\Cursos\Python\rh_django\rh_django__init__.py

```
# Habilita PyMySQL como reemplazo de MySQLdb
import pymysql
pymysql.install_as_MySQLdb()
```

5) Crear modelo Empleado

CAMBIOS IMPORTANTES

- Tabla empleado con idEmpleado (PK autoincremental), nombre, departamento, sueldo (decimal 10,2).

EXISTENTE — reemplazar contenido por el archivo completo:

Ruta:

C:\Cursos\Python\rh_django\empleados\models.py

```
from django.db import models

class Empleado(models.Model):
    idEmpleado = models.AutoField(primary_key=True)
    nombre = models.CharField(max_length=255) # requerido
    departamento = models.CharField(max_length=100) # requerido
    sueldo = models.DecimalField(max_digits=10, decimal_places=2) # > 0

    class Meta:
        db_table = 'empleado' # nombre exacto de la tabla

    def __str__(self):
        return f'{self.idEmpleado} - {self.nombre} ({self.departamento})'
```

6) Crear serializer con validaciones básicas

CAMBIOS IMPORTANTES

- Valida nombre y departamento no vacíos.
- Valida sueldo > 0.
- Expone idEmpleado como solo lectura.

NUEVO — crear archivo completo:

Ruta:

C:\Cursos\Python\rh_django\empleados\serializers.py

```
from rest_framework import serializers
from .models import Empleado

class EmpleadoSerializer(serializers.ModelSerializer):
    idEmpleado = serializers.IntegerField(read_only=True)

    class Meta:
        model = Empleado
        fields = ['idEmpleado', 'nombre', 'departamento', 'sueldo']
```

```

def validate_nombre(self, value: str):
    v = (value or '').strip()
    if not v:
        raise serializers.ValidationError("El nombre es requerido.")
    return v

def validate_departamento(self, value: str):
    v = (value or '').strip()
    if not v:
        raise serializers.ValidationError("El departamento es
requerido.")
    return v

def validate_sueldo(self, value):
    # Decimal o numérico; DRF ya valida tipo, aquí reforzamos la
regla > 0
    if value is None:
        raise serializers.ValidationError("El sueldo es requerido.")
    if value <= 0:
        raise serializers.ValidationError("El sueldo debe ser mayor a
0.")
    return value

```

7) Crear vistas con DRF generics

CAMBIOS IMPORTANTES

- ListCreateAPIView para listar/crear.
- RetrieveUpdateDestroyAPIView para ver/editar/eliminar.
- lookup_field implícito con pk → usa idEmpleado al ser PK.

EXISTENTE — reemplazar contenido por el archivo completo:

Ruta:

C:\Cursos\Python\rh_django\empleados\views.py

```

from rest_framework import generics
from .models import Empleado
from .serializers import EmpleadoSerializer

class EmpleadoListCreateAPIView(generics.ListCreateAPIView):
    queryset = Empleado.objects.all().order_by('idEmpleado')
    serializer_class = EmpleadoSerializer

class EmpleadoRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIVie
w):
    queryset = Empleado.objects.all()
    serializer_class = EmpleadoSerializer

```

```
# lookup_field = 'pk' # implícito; pk mapea al campo PK del modelo  
(idEmpleado)
```

8) Configurar URLs (app y proyecto)

CAMBIOS IMPORTANTES

- Endpoint base EXACTO requerido: `http://localhost:8080/api/empleados` (sin “/” final).
- Detalle por id: `http://localhost:8080/api/empleados/1`

NUEVO — crear archivo completo (urls de la app):

Ruta:

C:\Cursos\Python\rh_django\empleados\urls.py

```
from django.urls import path  
from .views import (  
    EmpleadoListCreateAPIView,  
    EmpleadoRetrieveUpdateDestroyAPIView,  
)  
  
urlpatterns = [  
    path('api/empleados', EmpleadoListCreateAPIView.as_view(),  
        name='empleado-list-create'),  
    path('api/empleados/<int:pk>',  
        EmpleadoRetrieveUpdateDestroyAPIView.as_view(), name='empleado-detail'),  
]
```

EXISTENTE — reemplazar contenido por el archivo completo (urls del proyecto):

Ruta:

C:\Cursos\Python\rh_django\rh_django\urls.py

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    # Incluimos las rutas de la app empleados en la raíz  
    path('', include('empleados.urls')),  
]
```

9) Crear base de datos y ejecutar migraciones

9.1 Crear base de datos en MySQL

- *Qué hace:* crea la BD con charset moderno.

Conéctate a MySQL:

```
mysql -u root -p
# (ingresa contraseña: admin)
```

En el prompt de MySQL:

```
CREATE DATABASE IF NOT EXISTS recursos_humanos_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
EXIT;
```

9.2 Generar migraciones

- *Qué hace:* crea archivos de migración a partir de modelos.

Desde la raíz del proyecto:

```
python manage.py makemigrations
```

9.3 Aplicar migraciones

- *Qué hace:* crea las tablas físicas en MySQL.

```
python manage.py migrate
```

10) Ejecutar servidor en puerto 8080

- *Qué hace:* levanta el servidor de desarrollo en el puerto requerido.

```
python manage.py runserver 8080
```

Debes ver el servidor en:
<http://localhost:8080/>

11) Pruebas con Postman (CRUD completo)

Colección de pruebas sugerida:**1. Listar empleados**

- Método: GET
- URL: `http://localhost:8080/api/empleados`
- Respuesta: 200 OK
- CORS: permitido desde `http://localhost:4200` y `http://localhost:5173`.

2. Crear empleado

- Método: POST
- URL: `http://localhost:8080/api/empleados`
- Headers: Content-Type: `application/json`
- Body (JSON):

```
3. {
4.   "nombre": "Carlos Lara",
5.   "departamento": "Sistemas",
6.   "sueldo": 25000.00
7. }
```

- Respuesta exitosa: 201 Created con JSON:

```
8. {
9.   "idEmpleado": 1,
10.  "nombre": "Carlos Lara",
11.  "departamento": "Sistemas",
12.  "sueldo": "25000.00"
13. }
```

- Errores de validación posibles (ejemplos):
 - `sueldo <= 0` → 400 Bad Request con `{"sueldo": ["El sueldo debe ser mayor a 0."]}`
 - nombre vacío → `{"nombre": ["El nombre es requerido."]}`

14. Buscar por id

- Método: GET
- URL: `http://localhost:8080/api/empleados/1`
- Respuesta: 200 OK o 404 Not Found si no existe.

15. Editar (reemplazo total)

- Método: PUT
- URL: `http://localhost:8080/api/empleados/1`
- Body (JSON):

```
16. {
17.   "nombre": "Carlos Lara",
18.   "departamento": "TI",
19.   "sueldo": 28000.00
20. }
```

- Respuesta: 200 OK con JSON actualizado.

21. Editar parcial

- Método: PATCH
- URL: `http://localhost:8080/api/empleados/1`
- Body (JSON):

```
22. { "sueldo": 30000.00 }
```

- Respuesta: 200 OK.

23. Eliminar

- Método: DELETE
- URL: `http://localhost:8080/api/empleados/1`
- Respuesta: 204 No Content.

Nota CORS (muy importante): Ya que configuramos `django-cors-headers` con:

- `CORS_ALLOWED_ORIGINS` → `http://localhost:4200` (Angular) y `http://localhost:5173` (React Vite)
 - Middleware en primer lugar
 - DRF sin autenticación (evita CSRF)
- Las peticiones desde front-ends en esos orígenes deben funcionar sin el error:
“No 'Access-Control-Allow-Origin' header is present on the requested resource.”

12) Estructura final del proyecto (archivos relevantes)

C:\Cursos\Python\rh_django	
.env\	(entorno virtual)
manage.py	
empleados\	
__init__.py	(EXISTENTE, sin cambios)
admin.py	(EXISTENTE, sin cambios)
apps.py	(EXISTENTE, sin cambios)
migrations\	(se generará 0001 tras
makemigrations)	
models.py	(EXISTENTE, CAMBIO: se definió
Empleado)	
serializers.py	(NUEVO, validaciones básicas)
urls.py	(NUEVO, rutas /api/empleados)
views.py	(EXISTENTE, CAMBIO: vistas
generics)	
rh_django\	
__init__.py	(EXISTENTE, CAMBIO: PyMySQL como
MySQLdb)	
asgi.py	(EXISTENTE, sin cambios)
settings.py	(EXISTENTE, CAMBIOS: MySQL, DRF,
CORS, app)	
urls.py	(EXISTENTE, CAMBIO: include
empleados.urls)	
wsgi.py	(EXISTENTE, sin cambios)

Resumen de “cambios importantes”

- **Conexión MySQL** por DATABASES y habilitación de **PyMySQL** en `__init__.py`.
 - **DRF sin autenticación/paginación** y permisos abiertos (`AllowAny`).
 - **CORS:**
 - `INSTALLED_APPS: 'corsheaders'`
 - `MIDDLEWARE: 'corsheaders.middleware.CorsMiddleware'` **antes de** `SecurityMiddleware`
 - `CORS_ALLOWED_ORIGINS: http://localhost:4200, http://localhost:5173`
 - **Métodos permitidos incluye** `OPTIONS` (para preflight).
 - **Modelo Empleado** con `idEmpleado` (PK), nombre, departamento, sueldo (10,2).
 - **Serializer** con validaciones (`> 0`, no vacío).
 - **Vistas generics** (`ListCreate / RetrieveUpdateDestroy`).
 - **URLs exactas:**
 - `GET/POST` → `http://localhost:8080/api/empleados`
 - `GET/PUT/PATCH/DELETE` → `http://localhost:8080/api/empleados/<id>`
-

Comandos finales de ejecución

1. Activar entorno:

```
..\venv\Scripts\activate
```

2. Levantar servidor en **8080**:

```
python manage.py runserver 8080
```

3. Probar en Postman (o desde React/Angular) los endpoints indicados.
-

¡Listo! Con esto tienes un **API REST Django** totalmente funcional, sobre **MySQL**, con **CORS** habilitado para **Angular** y **React (Vite)**, validaciones básicas y CRUD completo de empleados en `http://localhost:8080/api/empleados`.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)